🧩 Section 4 — Diving into Linux Namespaces

*Linux namespaces are one of the core mechanisms that make containers possible. They provide **per-container views** of global kernel resources.*

4.1 What Is a Namespace?

- A **namespace** provides an isolated instance of a particular global resource.
- Processes inside a namespace see **their own version** of:
  - process IDs
  - hostnames
  - network interfaces
  - mount points
  - IPC objects
  - user IDs, etc.
- Different namespaces can coexist on the same kernel.

Key idea:

> Namespaces do not create new kernels. They create new *views* of the same kernel-

4.2 Types of Namespaces Used by Containers

Common namespace types (especially relevant to Docker):

Namespace	Flag	Isolates
PID	`CLONE_NEWPID`	Process ID space, process tree
Mount	`CLONE_NEWNS`	Filesystem mount points, rootfs
UTS	`CLONE_NEWUTS`	Hostname and NIS domain name
Network	`CLONE_NEWNET`	Network interfaces, routing tables
IPC	`CLONE_NEWIPC`	SysV IPC, POSIX message queues
User	`CLONE_NEWUSER`	User and group IDs (UID/GID mapping)

Containers typically use **several namespaces together**, creating the illusion of a separate system.

4.3 PID Namespace

Goal: Isolate the process ID space.

- Inside a PID namespace:
  - Process IDs start at 1 (an "init"-like process).
  - Processes only see other processes in the same namespace.
- The host still sees all processes with **different PIDs**.

Docker example

Run an interactive container:

```
docker run --rm -it ubuntu bash
```

Inside the container:

```
ps aux
```

4.4 UTS Namespace (Hostname Isolation)

Goal: Isolate system identifiers like hostname.

- Each UTS namespace has its own:
  - hostname
  - NIS domain name

Inside a container:

```
hostname
```

On the host:

```
hostname
```

Observation:

4.5 Network Namespace

Goal: Isolate networking stack per container.

Each network namespace has:

- its own network interfaces
- its own routing table
- its own firewall rules
- its own `/proc/net` view

In a typical Docker setup:

- The host has a **bridge interface** (e.g., `docker0`).
- Each container gets a **veth (virtual Ethernet) interface**.
- The container sees only its own interfaces (e.g., `eth0`, `lo`).
- IP addresses inside the container are different from the host's

4.6 Mount Namespace (Filesystem View)

Goal: Provide independent views of the filesystem hierarchy.

- Each mount namespace has:
  - its own set of mounted filesystems
  - its own root directory (`/`)
- Unmounting or mounting a filesystem inside a namespace **does not affect** other namespaces.

In containers:

- The root filesystem comes from the container image (via OverlayFS or similar).
- Host paths are exposed only if explicitly **bound** (e.g., `–v /host/path:/container/path`).

Inside the container:

4.7 IPC Namespace

Goal: Isolate inter-process communication objects.

- Each IPC namespace has its own:
  - System V IPC objects (message queues, semaphores, shared memory)
  - POSIX message queues

This ensures that IPC used inside a container does not accidentally interact with IPC outside it.

Inside a container, IPC-related tools will show only the namespace-local objects.

OS concept illustrated:

> Synchronization and communication constructs can be scoped to a namespace.

4.8 User Namespace

Goal: Map user and group IDs inside the namespace to different IDs on the host.

- Allows **unprivileged** processes on the host to appear as **root (UID 0)** inside the namespace.
- The kernel maps container UID/GID to non-zero, unprivileged host UID/GID values.

This improves security:

- A process that appears to run as root in the container does **not** have root privileges on the host.

OS concept illustrated:

> Security and privilege can be virtualized independently of numeric UID/GID values.

4.9 Combining Namespaces

Containers typically combine several namespaces:

- PID namespace → process isolation
- Network namespace → network isolation
- Mount namespace → filesystem isolation
- UTS namespace → hostname isolation
- IPC namespace → IPC isolation
- User namespace → privilege isolation

Together, they create a **virtual environment** sharing a single kernel but appearing as a separate system.

Conceptual picture

4.10 Minimal Hands-On Namespace Experiment (Optional, Non-Docker)

For environments where `unshare` is available:

```
sudo unshare -p -f --mount-proc bash
```

Inside the new shell:

```
echo "Inside PID namespace:"
ps aux
echo "PID here:" $$
```

On the host:

```
echo "On host:"
ps aux | grep bash
```

4.11 Summary

- Namespaces are **kernel-level mechanisms** that provide isolated views of global resources.
- Containers are built by combining multiple namespaces:
  - PID, UTS, Network, Mount, IPC, User.
- These mechanisms allow:
  - process isolation
  - filesystem isolation
  - network isolation
  - privilege separation
- Namespaces demonstrate how the OS can present **multiple logical systems** on top of one physical kernel.